Job Search — Handoff técnico y funcional

1. Propósito de este documento

Este documento define el contexto, objetivos, arquitectura, alcance inicial, decisiones técnicas y plan de implementación de una nueva aplicación llamada provisionalmente **Job Search**.

Debe utilizarse como handoff para iniciar el proyecto con un agente de desarrollo en OpenCode.

El objetivo no es generar todo el sistema en una sola iteración. El proyecto debe construirse incrementalmente, con componentes pequeños, validaciones claras y una arquitectura preparada para evolucionar sin introducir infraestructura innecesaria.

---

2. Contexto general

2.1 Problema que queremos resolver

Queremos construir una aplicación personal que busque oportunidades laborales de software engineering, las normalice, elimine duplicados, descarte ofertas incompatibles y presente diariamente las oportunidades más relevantes.

La herramienta debe reducir el trabajo manual necesario para:

- revisar múltiples portales de empleo;
- abrir ofertas irrelevantes;
- detectar restricciones geográficas;
- comparar tecnologías requeridas;
- reconocer empleos duplicados;
- identificar oportunidades compatibles con el perfil;
- priorizar las mejores postulaciones;
- mantener un registro histórico de lo encontrado.

La aplicación no debe limitarse a devolver enlaces obtenidos desde un buscador. Debe convertirse en un pipeline persistente de adquisición, análisis, filtrado y ranking de oportunidades.

2.2 Referencia inicial

El punto de partida conceptual fue un workflow de n8n llamado `Job Scrapper`.

Ese workflow:

1. recibe o almacena un CV;
2. utiliza un modelo para convertirlo a una consulta de búsqueda;
3. ejecuta una única búsqueda mediante Tavily;
4. limita los resultados a dominios laborales;
5. toma hasta diez enlaces;

6. los envía mediante Telegram.

La idea de restringir la búsqueda a fuentes relevantes es útil, pero el workflow no realiza scraping real, persistencia, deduplicación, extracción completa, matching profundo ni seguimiento histórico.

Este nuevo proyecto no debe copiar literalmente ese workflow. Debe conservar su simplicidad conceptual y resolver sus limitaciones.

---

3. Infraestructura existente

3.1 Homelab

La aplicación será desplegada inicialmente en un homelab personal.

Características relevantes:

- Debian 13.
- CPU Ryzen 5 3600.
- 16 GB de RAM.
- GPU AMD RX 580 con 8 GB de VRAM.
- llama.cpp con backend Vulkan.
- Nginx como reverse proxy.
- Servicios administrados mediante systemd.
- Aplicaciones existentes basadas en FastAPI.
- YTPipe ya utiliza un modelo local para resumir videos de YouTube.

3.2 Modelos locales

Actualmente se utiliza:

```
Qwen3-4B-Instruct-2507-Q4_K_M
```

Este modelo se utiliza para tareas rápidas y estructuradas, especialmente dentro de YTPipe.

Se incorporará también:

```
Qwen3-4B-Thinking-2507-Q4_K_M
```

La estrategia definida es mantener dos modelos especializados, pero cargar solamente uno a la vez.

Modelo Instruct

Responsabilidades:

- YTPipe.
- Extracción estructurada de ofertas.

- Clasificación de ofertas.
- Normalización de campos.
- Resúmenes cortos.
- Identificación de requisitos y evidencia.
- Tareas que no necesitan razonamiento extenso.

Modelo Thinking

Responsabilidades:

- Evaluación profunda de ofertas finalistas.
- Análisis de compatibilidad entre experiencia y requisitos.
- Interpretación de requisitos ambiguos.
- Identificación de fortalezas, brechas y riesgos.
- Recomendación final de postulación.
- Revisión de casos con información contradictoria.

Inicialmente no se utilizará un modelo de 8B.

Un modelo mayor solamente será considerado si una evaluación controlada demuestra que el modelo Thinking de 4B falla significativamente en el ranking o en la interpretación de casos relevantes.

---

4. LLM Broker

4.1 Estado actual

Ya se inició otro proyecto independiente llamado provisionalmente **LLM Broker**.

Está siendo desarrollado en Go como ejercicio de aprendizaje y como componente reutilizable de infraestructura.

El broker debe ser agnóstico respecto de las aplicaciones consumidoras y los modelos concretos.

Sus responsabilidades incluyen:

- encolado de solicitudes;
- priorización;
- selección de modelo;
- model swapping;
- coordinación de workloads;
- control de concurrencia;
- interacción con llama.cpp o llama-swap;
- exposición de estado;
- prevención de cambios perjudiciales de modelo;
- aislamiento entre aplicaciones consumidoras.

4.2 Relación entre Job Search y el broker

Job Search no debe:

- iniciar llama-server;
- detener llama-server;
- reiniciar servicios de modelos;
- administrar directamente llama-swap;
- decidir cómo cargar modelos;
- conocer rutas locales de archivos GGUF;
- asumir qué modelo está cargado;
- implementar su propia cola de inferencia global.

Job Search debe consumir el LLM Broker como un servicio externo.

La aplicación podrá solicitar capacidades conceptuales como:

```
structured-extraction
deep-job-evaluation
short-summary
```

Idealmente, Job Search no debería depender directamente del nombre físico del modelo.

El broker decidirá qué modelo satisface cada capacidad.

Ejemplo conceptual:

```
{
  "application": "job-search",
  "capability": "structured-extraction",
  "priority": "normal",
  "payload": {
    "prompt": "...",
    "response_schema": {}
  }
}
```

Para evaluación profunda:

```
{
  "application": "job-search",
  "capability": "deep-job-evaluation",
  "priority": "batch",
  "payload": {
    "candidate_profile": {},
    "job": {},
    "preliminary_score": {}
```

```
  }
}
```

El contrato exacto debe adaptarse a la API que finalmente exponga el LLM Broker.

Debe existir una abstracción interna:

```python
class LLMProvider(Protocol):
    async def generate_structured(...): ...
    async def evaluate_deep_match(...): ...
```

Y una implementación:

```python
class LLMBrokerClient:
    ...
```

Esto permitirá probar Job Search con fixtures o un proveedor simulado sin requerir que el broker esté activo.

---

5. Decisión tecnológica

5.1 Lenguaje

La aplicación Job Search se desarrollará en Python.

Versión recomendada:

```
Python 3.13
```

5.2 Framework y librerías iniciales

Stack inicial:

```
FastAPI
Pydantic
SQLAlchemy
Alembic
httpx
SQLite
pytest
Ruff
mypy o Pyright
```

Se recomienda utilizar `uv` para administración del entorno, dependencias y ejecución de comandos.

5.3 Motivos para utilizar Python

Job Search será principalmente una aplicación de:

- ingestión de datos;
- parsing de HTML;
- normalización de texto;
- extracción estructurada;
- experimentación con prompts;
- evaluación de modelos;
- procesamiento batch;
- cálculo de métricas;
- análisis de datasets.

Python ofrece un ecosistema especialmente apropiado para estas tareas.

La división tecnológica queda conceptualmente así:

```
LLM Broker → Go
Job Search → Python
YTPipe → consumidor existente
Frontend futuro → React o React Native
```

5.4 Alternativas descartadas inicialmente

Go

Sería viable, pero ya se está utilizando en el broker. Python ofrece mayor velocidad de experimentación para extracción, scoring y evaluación de modelos.

TypeScript y NestJS

Sería una alternativa válida, pero aporta menos valor para procesamiento de datos y evaluación local de LLMs.

Django

Podría ser útil si el proyecto necesitara inmediatamente un panel administrativo completo, autenticación y múltiples usuarios. Para esta primera etapa introduciría demasiadas convenciones y componentes.

Rust

No se justifica para una aplicación experimental cuyo dominio todavía cambiará frecuentemente.

---

6. Principios arquitectónicos

6.1 API, worker y scheduler separados

FastAPI será la interfaz HTTP, pero no ejecutará directamente el batch completo.

La aplicación tendrá tres procesos conceptuales:

```
API
Worker
Scheduler
```

API

Responsabilidades:

- consultar ofertas;
- consultar ejecuciones;
- administrar el perfil;
- cambiar estados;
- registrar feedback;
- iniciar una ejecución manual;
- exponer salud y métricas básicas.

Worker

Responsabilidades:

- recolectar ofertas;
- normalizar resultados;
- deduplicar;
- ejecutar extracción;
- calcular scores;
- solicitar evaluaciones al broker;
- persistir resultados;
- preparar notificaciones.

Scheduler

Responsabilidad:

- crear ejecuciones programadas.

Inicialmente se recomienda utilizar un `systemd timer` que invoque un comando CLI de la aplicación.

Ejemplo conceptual:

```
job-search run create --reason scheduled
```

No se utilizará `FastAPI BackgroundTasks` para ejecutar el pipeline completo.

6.2 Procesamiento persistente

Una ejecución no debe depender de mantener abierta una petición HTTP.

La API debe poder responder:

```
{
  "run_id": "run_123",
  "status": "pending"
}
```

Luego el worker procesa la ejecución independientemente.

Si el worker o la API se reinician, la ejecución debe poder continuar o reintentarse.

6.3 Una sola máquina y un solo worker inicialmente

No se necesita una arquitectura distribuida.

Primera versión:

- un único servidor;
- un único worker;
- una única base SQLite;
- un único scheduler;
- broker externo para inferencia.

No introducir inicialmente:

- Celery;
- Redis;
- Kafka;
- RabbitMQ;
- Kubernetes;
- múltiples workers;
- locking distribuido.

---

7. Estructura propuesta del repositorio

```
job-search/
├── app/
```

```
│   ├── api/
│   │   ├── dependencies.py
│   │   ├── health.py
│   │   ├── jobs.py
│   │   ├── profiles.py
│   │   ├── runs.py
│   │   └── sources.py
│   │
│   ├── broker/
│   │   ├── client.py
│   │   ├── schemas.py
│   │   └── protocol.py
│   │
│   ├── collectors/
│   │   ├── base.py
│   │   ├── greenhouse.py
│   │   ├── lever.py
│   │   ├── ashby.py
│   │   ├── workable.py
│   │   ├── web_search.py
│   │   └── generic_web.py
│   │
│   ├── extraction/
│   │   ├── extractor.py
│   │   ├── prompts.py
│   │   ├── schemas.py
│   │   └── validation.py
│   │
│   ├── matching/
│   │   ├── hard_filters.py
│   │   ├── preliminary_score.py
│   │   ├── deep_evaluation.py
│   │   ├── policies.py
│   │   └── schemas.py
│   │
│   ├── notifications/
│   │   ├── base.py
│   │   └── telegram.py
│   │
│   ├── repositories/
│   │   ├── jobs.py
│   │   ├── profiles.py
│   │   ├── runs.py
│   │   └── feedback.py
│   │
│   ├── services/
│   │   ├── collection_service.py
│   │   ├── deduplication_service.py
│   │   ├── extraction_service.py
│   │   ├── matching_service.py
```

```
│   │   ├── notification_service.py
│   │   └── run_service.py
│   │
│   ├── database/
│   │   ├── base.py
│   │   ├── models.py
│   │   └── session.py
│   │
│   ├── domain/
│   │   ├── candidate.py
│   │   ├── job.py
│   │   ├── run.py
│   │   └── source.py
│   │
│   ├── cli/
│   │   └── main.py
│   │
│   ├── config.py
│   ├── logging.py
│   └── main.py
│
├── worker/
│   └── main.py
│
├── migrations/
├── prompts/
├── fixtures/
├── evaluation/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/
│
├── pyproject.toml
├── alembic.ini
├── README.md
├── .env.example
└── Makefile
```

Esta estructura representa el objetivo arquitectónico. No es necesario crear todos los módulos vacíos desde el primer commit.

Se debe comenzar con una estructura mínima y agregar módulos cuando tengan una responsabilidad real.

---

8. Perfil del candidato

8.1 Perfil normalizado

La aplicación no debe enviar el CV completo a los modelos en cada ejecución.

El CV debe convertirse una vez en un perfil estructurado.

Ejemplo:

```
{
  "profile_id": "default",
  "title": "Senior Frontend Engineer",
  "years_of_experience": 8,
  "target_roles": [
    "frontend",
    "fullstack"
  ],
  "target_titles": [
    "Senior Frontend Engineer",
    "Senior React Developer",
    "Senior Software Engineer",
    "Full Stack Engineer"
  ],
  "core_skills": [
    "React",
    "TypeScript",
    "JavaScript"
  ],
  "secondary_skills": [
    "Node.js",
    "Next.js"
  ],
  "regions": [
    "Argentina",
    "LATAM",
    "Latin America",
    "Americas",
    "Worldwide"
  ],
  "remote_only": true,
  "employment_preferences": [
    "full_time",
    "contractor"
  ],
  "languages": {
    "spanish": "native",
    "english": "professional"
  },
```

```
  "excluded_roles": [],
  "excluded_technologies": [],
  "hard_constraints": {},
  "soft_preferences": {}
}
```

8.2 Datos personales

Nombre, teléfono, email y otra información sensible no son necesarios para evaluar una oferta.

El matching debe trabajar con un perfil profesional reducido.

El CV completo solo será necesario más adelante para:

- adaptar una postulación;
- redactar una presentación;
- preparar una respuesta;
- generar una versión específica del CV.

Estas funciones no forman parte del MVP.

---

9. Fuentes de ofertas

La aplicación debe utilizar una arquitectura de collectors.

9.1 Interfaz conceptual

```python
class JobCollector(Protocol):
    source_name: str

    async def collect(self, context: CollectionContext) -> list[RawJob]:
        ...
```

Cada collector devuelve un formato interno común:

```
{
  "source": "greenhouse",
  "external_id": "12345",
  "source_url": "...",
  "application_url": "...",
  "title": "...",
  "company": "...",
  "location": "...",
  "description": "...",
  "structured_data": {},
```

```
  "collected_at": "..."
}
```

9.2 Fuentes prioritarias

Orden inicial recomendado:

1. Greenhouse.
2. Lever.
3. Ashby.
4. Workable.
5. Búsqueda web mediante proveedor configurable.
6. Páginas genéricas de empresas.
7. Alertas de LinkedIn por correo.

9.3 ATS estructurados

Cuando un ATS entrega campos estructurados, esos campos deben considerarse preferibles al resultado del LLM.

Ejemplos:

- título;
- empresa;
- ubicación;
- ID externo;
- URL de aplicación;
- departamento;
- fecha de publicación.

El LLM no debe redescubrir información que la fuente ya proporciona correctamente.

Debe utilizarse para extraer información que permanece dentro de la descripción:

- requisitos;
- skills obligatorias;
- skills deseables;
- autorización laboral;
- idiomas;
- modalidad contractual;
- salario si aparece en texto;
- restricciones regionales;
- años de experiencia.

9.4 Búsqueda web

La búsqueda web será una fuente de descubrimiento, no una fuente de verdad.

Debe existir una abstracción:

```python
class SearchProvider(Protocol):
    async def search(self, query: SearchQuery) -> list[SearchResult]:
        ...
```

Tavily puede ser una implementación inicial, pero no debe estar acoplado al dominio.

Las búsquedas deben poder restringirse a portales como:

- Greenhouse;
- Lever;
- Ashby;
- Workable;
- Wellfound;
- Work at a Startup;
- Workana;
- Get on Board;
- Revelo;
- LinkedIn.

No se debe generar una única consulta demasiado restrictiva.

Debe existir un conjunto controlado de consultas por:

- rol;
- título;
- stack;
- región;
- modalidad contractual.

9.5 LinkedIn

No se debe implementar scraping autenticado de LinkedIn.

Queda explícitamente fuera del alcance:

- utilizar credenciales del usuario;
- automatizar login;
- conservar cookies;
- utilizar Playwright con una sesión personal;
- eludir CAPTCHA;
- simular comportamiento humano;
- hacer crawling de resultados autenticados.

Las vías aceptables son:

Descubrimiento mediante buscadores

Buscar páginas públicas indexadas del dominio LinkedIn.

Ejemplo conceptual:

```
site:linkedin.com/jobs/view senior React remote LATAM
```

Esta fuente será parcial y no debe considerarse exhaustiva.

Alertas oficiales por correo

LinkedIn puede realizar la búsqueda mediante sus propias alertas.

En una fase posterior, Job Search podrá procesar los emails recibidos:

```
LinkedIn Job Alert
→ mailbox
→ extracción de enlaces
→ normalización
→ deduplicación
→ análisis
```

Resolución hacia el ATS original

Cuando una oferta de LinkedIn enlaza a Greenhouse, Lever, Ashby, Workable o la página de la empresa, debe preferirse la fuente original.

---

10. Pipeline completo

10.1 Inicio de la ejecución

Una ejecución puede comenzar mediante:

- scheduler;
- API;
- CLI;
- reintento manual.

Se crea un registro `JobSearchRun`.

Estados posibles:

```
pending
collecting
normalizing
extracting
scoring
deep_evaluating
notifying
completed
```

```
failed
cancelled
```

10.2 Recolección

Los collectors configurados obtienen ofertas.

Cada resultado se persiste inicialmente como `RawJob`.

Debe conservarse suficiente información para poder reproducir o depurar el procesamiento.

10.3 Normalización

La normalización incluye:

- limpieza de URLs;
- eliminación de parámetros de tracking;
- normalización de nombres de empresa;
- normalización de espacios;
- conversión de HTML a texto;
- detección de IDs externos;
- normalización de ubicaciones;
- normalización básica de títulos.

10.4 Deduplicación

La deduplicación debe utilizar varias señales, en orden:

1. fuente + ID externo;
2. URL de aplicación normalizada;
3. URL de fuente normalizada;
4. empresa + título + ubicación;
5. fingerprint de empresa, título y descripción.

Una misma oferta puede aparecer mediante:

- LinkedIn;
- Tavily;
- página de careers;
- ATS;
- email.

Debe mantenerse una entidad canónica y varias referencias de origen.

10.5 Prefiltros determinísticos

Antes de usar el modelo pueden descartarse casos evidentes:

- rol completamente ajeno;

- ubicación presencial incompatible;
- oferta eliminada;
- URL inválida;
- contenido vacío;
- duplicado ya procesado;
- país explícitamente incompatible;
- tecnología central expresamente excluida.

Los prefiltros deben ser conservadores.

Es preferible mantener temporalmente una oferta dudosa que eliminar una oportunidad valiosa demasiado pronto.

10.6 Extracción estructurada con Instruct

El modelo Instruct recibe:

- campos confiables entregados por la fuente;
- descripción limpia;
- campos concretos que debe extraer;
- JSON Schema esperado.

No debe recibir HTML completo si ya se obtuvo el texto relevante.

Salida conceptual:

```
{
  "seniority": {
    "value": "senior",
    "evidence": "We are looking for a Senior Frontend Engineer..."
  },
  "remote_policy": {
    "value": "remote",
    "evidence": "This is a fully remote position."
  },
  "regions": {
    "value": ["latin_america"],
    "evidence": "Candidates must be located in Latin America."
  },
  "employment_type": {
    "value": "contractor",
    "evidence": "This is a long-term contractor position."
  },
  "required_skills": [
    {
      "value": "React",
      "evidence": "5+ years working with React."
    }
  ],
  "preferred_skills": [
```

```
    {
      "value": "GraphQL",
      "evidence": "GraphQL experience is a plus."
    }
  ],
  "languages": [
    {
      "value": "English",
      "level": "advanced",
      "evidence": "Excellent spoken and written English."
    }
  ],
  "years_experience": {
    "value": 5,
    "evidence": "5+ years of professional experience."
  },
  "salary": null,
  "work_authorization": {
    "value": "not_specified",
    "evidence": null
  }
}
```

10.7 Validación de la extracción

La salida se valida mediante:

1. respuesta restringida mediante JSON Schema;
2. validación sintáctica;
3. `Pydantic.model_validate_json`;
4. enums y tipos;
5. validaciones semánticas;
6. comprobación de evidencia;
7. un único reintento correctivo si la respuesta es inválida.

No se deben realizar reintentos indefinidos.

Si vuelve a fallar:

```
extraction_status = needs_review
```

10.8 Scoring preliminar

El score inicial debe calcularse en código.

No se debe pedir al modelo Instruct un número arbitrario de compatibilidad.

Señales posibles:

- compatibilidad de rol;
- seniority;
- región;
- modalidad remota;
- tipo de contratación;
- skills centrales;
- skills secundarias;
- experiencia requerida;
- idioma;
- salario;
- industria;
- preferencias personales.

Ejemplo conceptual:

```
role_match              0–20
location_match          0–20
core_skill_match        0–25
secondary_skill_match   0–10
seniority_match         0–10
employment_match        0–10
language_match          0–5
```

El peso exacto no debe fijarse prematuramente.

La implementación debe utilizar señales individuales y configurables, no una función monolítica imposible de depurar.

Salida:

```
{
  "preliminary_score": 78,
  "signals": [
    {
      "name": "location_match",
      "score": 20,
      "reason": "Open to candidates in Latin America."
    },
    {
      "name": "core_skill_match",
      "score": 21,
      "reason": "React and TypeScript are required."
    }
  ],
  "hard_blockers": [],
  "ambiguities": [
    "Employment type is not explicit."
```

```
  ]
}
```

10.9 Selección para Thinking

No todas las ofertas deben pasar por el modelo Thinking.

Candidatos posibles:

- score preliminar alto;
- oferta interesante con requisitos ambiguos;
- conflicto entre reglas;
- región poco clara;
- seniority ambiguo;
- skill principal transferible pero no exacta;
- puesto con buen encaje general y un posible bloqueo;
- ofertas cercanas al umbral de decisión.

Los thresholds deben ser configurables.

Ejemplo inicial:

```
score >= 75 → deep evaluation
55–74 con ambigüedades → deep evaluation
score < 55 sin señales especiales → no deep evaluation
hard blocker seguro → no deep evaluation
```

10.10 Procesamiento agrupado por modelo

El orden debe evitar alternar modelos por cada oferta.

Secuencia:

1. recolectar todas las ofertas;
2. normalizar;
3. deduplicar;
4. aplicar filtros;
5. procesar todas las extracciones con Instruct;
6. persistir todos los JSON;
7. calcular todos los scores preliminares;
8. construir la cola de evaluación profunda;
9. solicitar Thinking para toda esa cola;
10. volver a dejar disponible Instruct.

El LLM Broker debe coordinar este comportamiento.

Job Search no debe implementar el swap directamente.

10.11 Evaluación profunda

Thinking recibe solamente:

- perfil profesional normalizado;
- extracción estructurada;
- evidencia relevante;
- score preliminar;
- señales que produjeron el score;
- ambigüedades;
- preguntas concretas.

No debe recibir nuevamente toda la página HTML.

Salida esperada:

```
{
  "fit_score": 82,
  "decision": "strong_match",
  "strengths": [
    "React and TypeScript are central requirements.",
    "The position is explicitly open to Latin America."
  ],
  "gaps": [
    "GraphQL experience is preferred but not required."
  ],
  "hard_blockers": [],
  "uncertainties": [
    "The contract type is not explicitly stated."
  ],
  "recommended_action": "apply",
  "confidence": 0.84
}
```

Valores sugeridos:

```
decision:
- strong_match
- possible_match
- weak_match
- reject
- needs_review

recommended_action:
- apply
- review
- ignore
```

10.12 Regla final

El modelo nunca puede ignorar un hard blocker confirmado por código.

Ejemplos:

- requiere residencia en Estados Unidos;
- requiere autorización laboral no disponible;
- es presencial en una región incompatible;
- no acepta candidatos desde Argentina;
- exige una tecnología central excluyente;
- modalidad incompatible definida como restricción dura.

Se recomienda mantener:

```
preliminary_score
thinking_score
thinking_adjustment
final_score
```

El ajuste del modelo puede limitarse inicialmente a un rango, por ejemplo:

```
-15 a +15
```

El score final siempre debe ser calculado por código.

---

11. Persistencia y modelo de datos

11.1 CandidateProfile

Campos principales:

```
id
name
version
profile_json
is_active
created_at
updated_at
```

11.2 Source

```
id
type
```

```
name
configuration
enabled
last_success_at
last_error_at
```

11.3 Job

Entidad canónica:

```
id
title
company
location
normalized_location
description
application_url
status
first_seen_at
last_seen_at
published_at
expires_at
created_at
updated_at
```

11.4 JobSourceReference

```
id
job_id
source_id
external_id
source_url
raw_payload
collected_at
```

11.5 JobExtraction

```
id
job_id
schema_version
prompt_version
model_capability
model_identifier
extraction_json
validation_status
```

```
attempt_count
created_at
```

11.6 JobScore

```
id
job_id
profile_id
scoring_version
preliminary_score
signal_json
hard_blocker_json
ambiguity_json
created_at
```

11.7 JobDeepEvaluation

```
id
job_id
profile_id
prompt_version
model_identifier
fit_score
decision
strengths
gaps
hard_blockers
uncertainties
recommended_action
confidence
created_at
```

11.8 JobSearchRun

```
id
reason
status
started_at
completed_at
error_message
stats_json
created_at
```

11.9 RunJob

Relaciona ofertas con una ejecución:

```
run_id
job_id
stage
status
error_message
created_at
updated_at
```

11.10 Notification

```
id
job_id
channel
status
message
sent_at
error_message
```

11.11 Feedback

```
id
job_id
profile_id
action
reason
notes
created_at
```

Acciones iniciales:

```
interesting
applied
ignored
not_relevant
duplicate
expired
```

11.12 SQLite

SQLite es suficiente inicialmente.

Configurar:

- WAL mode;
- foreign keys;
- busy timeout;
- migraciones con Alembic;
- un único worker escritor.

La capa de repositorios no debe depender de características exclusivas de SQLite, para permitir una migración futura a PostgreSQL.

---

12. API inicial

Salud

```
GET /health
```

Perfil

```
GET    /profiles
GET    /profiles/{id}
POST   /profiles
PUT    /profiles/{id}
```

Ejecuciones

```
POST   /runs
GET    /runs
GET    /runs/{id}
POST   /runs/{id}/retry
```

Ofertas

```
GET    /jobs
GET    /jobs/{id}
PATCH  /jobs/{id}/status
POST   /jobs/{id}/feedback
```

Filtros útiles:

```
minimum_score
decision
```

```
source
company
status
seen_after
requires_review
```

Fuentes

```
GET    /sources
PATCH  /sources/{id}
```

Métricas iniciales

```
GET /metrics/summary
```

No es necesario implementar todos los endpoints en la primera fase.

---

13. Notificaciones por Telegram

Telegram será la primera interfaz activa del producto.

El mensaje no debe limitarse a título y URL.

Ejemplo:

```
🟢 87/100 — Senior Frontend Engineer

🏢 Example Company
🌎 Remote — Latin America
💼 Contractor
🧰 React, TypeScript, Next.js

Por qué encaja:
• React y TypeScript son requisitos centrales.
• La posición acepta candidatos de LATAM.
• El seniority coincide con el perfil.

Atención:
• GraphQL aparece como requisito deseable.
• No se especifica el rango salarial.

🔗 Ver oferta
```

En una fase posterior podrán agregarse acciones:

```
✅ Me interesa
📨 Me postulé
❌ No relevante
🔕 Ignorar empresa
```

La notificación diaria debe priorizar calidad sobre cantidad.

El objetivo no es enviar todas las ofertas, sino las oportunidades que realmente merecen atención.

---

14. Configuración de modelos

Instruct

Uso:

- extracción estructurada;
- normalización;
- clasificación;
- resumen corto.

Configuración inicial sugerida:

```
temperature: baja
concurrency: 1
structured output: JSON Schema
retry: máximo 1
```

Los valores exactos deben medirse con el modelo real.

Thinking

Configuración inicial conservadora:

```
context: 8k o 12k
temperature: probar 0.3 y 0.6
top_p: 0.95
top_k: 20
max output: 2k–3k
timeout: 600 segundos
concurrency: 1
reasoning format: habilitado si el broker lo soporta
```

No almacenar el razonamiento interno.

Persistir únicamente:

- decisión;
- explicación breve;
- evidencia;
- score;
- confianza;
- modelo;
- versión del prompt;
- fecha.

Solamente aumentar contexto o salida si las pruebas demuestran que es necesario.

---

15. Prompts y versionado

Los prompts no deben estar dispersos como strings arbitrarios dentro de servicios.

Deben:

- almacenarse en archivos dedicados;
- tener una versión;
- incluir el schema esperado;
- ser testeables;
- registrar qué versión produjo cada resultado.

Ejemplo:

```
prompts/
├── job_extraction_v1.md
├── job_extraction_repair_v1.md
└── deep_match_v1.md
```

Cada resultado persistido debe guardar:

```
prompt_version
schema_version
model_identifier
```

Esto permitirá comparar resultados después de modificar un prompt.

---

16. Evaluación de calidad

Antes de confiar en el ranking, se debe preparar un dataset manual de entre 50 y 100 ofertas.

Cada oferta debe tener etiquetas humanas como:

```
interesting
not_interesting
hard_blocker
possible_match
strong_match
```

También deben señalarse:

- restricciones geográficas;
- modalidad;
- skills principales;
- requisitos ambiguos;
- decisión recomendada.

Métricas prioritarias

Hard-blocker recall

De todas las ofertas imposibles:

```
¿Cuántas fueron bloqueadas correctamente?
```

Interesting-job recall

De todas las ofertas interesantes:

```
¿Cuántas sobrevivieron al pipeline?
```

Top-10 precision

De las diez primeras:

```
¿Cuántas realmente merecían atención?
```

Ranking stability

Ejecutando varias veces la misma entrada:

```
¿Cuánto cambia el orden?
```

Extracción estructurada

Medir:

- exactitud de ubicación;

- exactitud de modalidad;
- exactitud de skills;
- exactitud de autorización laboral;
- porcentaje de JSON válido;
- porcentaje de reintentos;
- evidencia incorrecta o inventada.

Valor marginal de Thinking

Comparar:

```
Reglas solamente
Reglas + Instruct
Reglas + Thinking
Reglas + modelo mayor, solo si se prueba
```

La pregunta decisiva es:

```
¿Cuántas veces Thinking mejora correctamente una decisión?
```

No debe incorporarse un modelo de 8B sin demostrar una mejora significativa.

---

17. Seguridad y privacidad

Secretos

Utilizar variables de entorno para:

- URL del broker;
- API keys;
- token de Telegram;
- credenciales de buscadores;
- credenciales futuras de correo.

Nunca versionar secretos.

CV y perfil

No almacenar información personal innecesaria.

No registrar en logs:

- teléfono;
- email;
- CV completo;
- tokens;

- credenciales;
- contenido completo de solicitudes al modelo.

Fetching de URLs

Si la aplicación permite procesar URLs arbitrarias, debe considerar protección contra SSRF.

Inicialmente se recomienda:

- allowlist de esquemas `http` y `https`;
- bloquear IPs privadas y loopback;
- limitar redirecciones;
- limitar tamaño de respuesta;
- configurar timeouts;
- validar content type;
- aplicar rate limiting por dominio.

Scraping

Priorizar:

- APIs públicas;
- feeds;
- JSON-LD;
- páginas públicas;
- alertas oficiales.

No implementar mecanismos para eludir controles de acceso.

---

18. Observabilidad

Todos los logs deben ser estructurados.

Campos recomendados:

```
run_id
job_id
source
stage
model_capability
duration_ms
attempt
status
```

Métricas iniciales:

- ofertas recolectadas;

- ofertas nuevas;
- duplicados;
- ofertas descartadas;
- extracciones válidas;
- extracciones fallidas;
- evaluaciones Thinking;
- tiempo por etapa;
- tiempo esperando al broker;
- tokens si el broker los informa;
- notificaciones enviadas;
- errores por fuente.

Cada ejecución debe terminar con un resumen:

```
{
  "collected": 83,
  "new_jobs": 26,
  "duplicates": 41,
  "prefiltered": 8,
  "extracted": 18,
  "deep_evaluated": 5,
  "notified": 4,
  "failed": 1
}
```

---

19. Decisiones explícitamente descartadas

No implementar inicialmente:

- n8n como motor principal;
- LangChain;
- frameworks de agentes;
- base vectorial;
- RAG;
- Celery;
- Redis;
- Kafka;
- múltiples workers;
- frontend web;
- React Native;
- autoaplicación;
- generación automática de CV;
- scraping autenticado de LinkedIn;
- navegador automatizado con credenciales;
- modelo de 8B;
- clasificación completamente delegada al LLM;
- score generado únicamente por IA;
- almacenamiento del chain of thought;

- procesamiento del batch dentro de una request HTTP;
- reinicio directo de llama-server desde Job Search.

---

20. Fases de implementación

Fase 0 — Bootstrap

Objetivo:

Crear un proyecto Python mínimo y sano.

Entregables:

- `pyproject.toml`;
- entorno administrado con `uv`;
- FastAPI;
- configuración con Pydantic Settings;
- logging estructurado;
- Ruff;
- type checking;
- pytest;
- endpoint `/health`;
- README inicial;
- `.env.example`;
- estructura mínima del repositorio.

No crear todavía todos los módulos futuros.

Fase 1 — Dominio y persistencia

Objetivo:

Representar perfiles, ofertas y ejecuciones.

Entregables:

- modelos Pydantic;
- modelos SQLAlchemy;
- SQLite;
- Alembic;
- repositorios;
- `CandidateProfile`;
- `Job`;
- `JobSourceReference`;
- `JobSearchRun`;
- pruebas unitarias;
- fixtures.

Debe ser posible insertar una oferta fixture y consultarla mediante API.

Fase 2 — Primer collector

Objetivo:

Completar un camino de adquisición sin LLM.

Fuente inicial recomendada:

```
Greenhouse
```

Entregables:

- interfaz `JobCollector`;
- implementación Greenhouse;
- normalización;
- deduplicación;
- persistencia;
- ejecución manual mediante CLI;
- registro de estadísticas.

Criterio de éxito:

```
collector → normalized job → database → API
```

Fase 3 — Integración con LLM Broker

Objetivo:

Extraer información estructurada mediante Instruct.

Entregables:

- abstracción `LLMProvider`;
- cliente del broker;
- proveedor fake para tests;
- schema de extracción;
- prompt versionado;
- validación Pydantic;
- un reintento correctivo;
- persistencia de resultados.

Criterio de éxito:

Procesar fixtures y obtener JSON válido con evidencia.

Fase 4 — Hard filters y scoring

Objetivo:

Crear un ranking preliminar reproducible.

Entregables:

- perfil normalizado;
- hard filters;
- señales de scoring;
- score preliminar;
- razones del score;
- configuración de thresholds;
- tests de reglas.

Fase 5 — Ejecución persistente

Objetivo:

Separar API y worker.

Entregables:

- worker;
- estados de ejecución;
- recuperación tras reinicio;
- retry de etapas;
- CLI;
- systemd service;
- systemd timer.

Fase 6 — Thinking

Objetivo:

Analizar solamente casos altos o ambiguos.

Entregables:

- cola lógica de candidatos;
- prompt Thinking;
- schema final;
- integración con broker;
- procesamiento agrupado;
- score final determinístico;
- persistencia;
- métricas.

Fase 7 — Telegram

Objetivo:

Enviar un digest útil.

Entregables:

- formateador;
- cliente Telegram;
- top diario;
- registro de notificaciones;
- prevención de envíos duplicados.

Fase 8 — Más fuentes

Agregar progresivamente:

- Lever;
- Ashby;
- Workable;
- proveedor de búsqueda web;
- páginas genéricas.

No agregar todas simultáneamente.

Fase 9 — LinkedIn mediante fuentes seguras

Agregar:

- resultados públicos encontrados por buscador;
- procesamiento de alertas oficiales de LinkedIn por email;
- resolución hacia ATS original.

Fase 10 — Evaluación controlada

Crear dataset manual y comparar:

- reglas;
- Instruct;
- Thinking;
- configuraciones;
- prompts;
- thresholds.

---

21. Primer milestone recomendado

El primer milestone no debe incluir Telegram ni Thinking.

Debe demostrar este recorrido:

```
perfil fixture
→ ejecución manual
→ collector Greenhouse
→ normalización
→ deduplicación
→ SQLite
→ consulta mediante API
```

Luego:

```
oferta almacenada
→ LLM Broker mock
→ extracción validada
→ resultado persistido
```

Definition of Done

- El proyecto se instala mediante `uv sync`.
- `uv run pytest` pasa.
- `uv run ruff check .` pasa.
- El type checker pasa.
- `/health` responde correctamente.
- Se puede crear una ejecución manual.
- El worker procesa una ejecución.
- Un collector fixture o Greenhouse devuelve ofertas.
- Las ofertas se guardan sin duplicados.
- Los errores quedan registrados.
- La API permite consultar ejecución y ofertas.
- El diseño no depende todavía de Telegram, Tavily ni Thinking.

---

22. Instrucciones iniciales para el agente de OpenCode

1. Revisar este handoff completo.
2. Revisar el estado y contrato actual del LLM Broker.
3. No modificar el LLM Broker durante el bootstrap de Job Search salvo que exista una incompatibilidad real.
4. Crear un repositorio nuevo para Job Search.
5. Utilizar Python 3.13 y `uv`.
6. Implementar solamente la Fase 0.
7. Mostrar la estructura propuesta antes de crear una arquitectura excesiva.
8. Mantener el primer commit pequeño.
9. Agregar tests desde el inicio.

10. No introducir frameworks de agentes.
11. No implementar scraping de LinkedIn.
12. No agregar Redis, Celery ni infraestructura distribuida.
13. No utilizar todavía el modelo Thinking.
14. No construir todos los collectors.
15. Documentar cada decisión importante en el README o en ADRs pequeños.
16. Validar cada fase antes de continuar.

El enfoque de trabajo debe ser incremental:

```
concepto
→ implementación pequeña
→ validación
→ siguiente componente
```

No generar una aplicación completa y enorme en una única ejecución.

---

23. Visión de largo plazo

El producto podrá evolucionar hacia:

- digest diario;
- feedback desde Telegram;
- seguimiento de postulaciones;
- preparación asistida de candidatura;
- comparación entre distintas versiones de CV;
- historial de empresas;
- estadísticas del mercado;
- identificación de tecnologías demandadas;
- aplicación móvil en React Native;
- integración con un RAG personal;
- conversación con la IA local;
- búsqueda manual desde Telegram;
- explicación de por qué una oferta fue descartada;
- recomendaciones sobre qué skills mejorar.

Estas funciones no deben condicionar el diseño inicial más allá de mantener fronteras claras entre:

```
collectors
normalización
persistencia
matching
LLM Broker
notificaciones
API
```

El objetivo inmediato es construir un pipeline confiable, observable y evaluable que encuentre mejores oportunidades con menos revisión manual.